

IF25-22017 PENGEMBANGAN APLIKASI MOBILE

## Modul Belajar 7

# Penyimpanan Lokal dan Offline-first

Menyimpan laporan di perangkat dengan SQLDelight, menampilkan data terakhir saat offline, dan menjaga draft form

**Muhammad Habib Algifari, S.Kom., M.T.I.**

Pendamping Pertemuan 7. Tahun 2026

| Komponen      | Keterangan                                                                                                                 |
|---------------|----------------------------------------------------------------------------------------------------------------------------|
| Pertemuan     | 7                                                                                                                          |
| Capaian       | CPMK0501, menerapkan konsep pemrograman dalam bentuk aplikasi yang menyimpan data lokal dan tetap berfungsi tanpa internet |
| Waktu belajar | sekitar 5 jam belajar mandiri, ditambah 2,5 jam di kelas                                                                   |

## MODUL 7

# Peta belajar

Ikuti urutan ini dari atas ke bawah. Setiap bagian memakai apa yang sudah dibahas di bagian sebelumnya.

| Urutan | Bagian                                                 |
|--------|--------------------------------------------------------|
| 1      | Tentang modul ini                                      |
| 2      | Masalah pembuka: sinyal hilang, laporan ikut hilang    |
| 3      | Langkah 1: Tiga tempat menyimpan di perangkat          |
| 4      | Langkah 2: Preferensi, satu kunci dan satu nilai       |
| 5      | Langkah 3: Memasang SQLDelight                         |
| 6      | Langkah 4: Berkas .sq, tabel dan query                 |
| 7      | Langkah 5: Dari .sq menjadi Kotlin                     |
| 8      | Langkah 6: Driver per platform lewat expect dan actual |
| 9      | Langkah 7: Membuat database dan memanggil query        |
| 10     | Langkah 8: Prinsip offline-first                       |
| 11     | Langkah 9: Baca lokal, segarkan dari server            |
| 12     | Langkah 10: Draf form tersimpan otomatis               |
| 13     | Latihan mandiri                                        |
| 14     | Bawa ke proyekmu                                       |
| 15     | Rangkuman                                              |
| 16     | Cek pemahaman                                          |
| 17     | Bacaan lanjut dan persiapan minggu depan               |

## PEMBUKA

## Tentang modul ini

Modul ini menemani Pertemuan 7. Di Modul 6, Laporkampus sudah mengambil laporan dari server. Tetapi begitu internet hilang, yang tampil hanya pesan gagal, dan draf yang sedang ditulis ikut hilang saat aplikasi ditutup. Di modul ini kita menyimpan data di perangkat, supaya aplikasi tetap berguna walau sinyal di gedung kuliah sedang buruk.

Modul ini terdiri dari tiga bagian. Langkah 1 dan 2 membahas pilihan tempat menyimpan data di perangkat. Langkah 3 sampai 7 membahas **SQLDelight**, pustaka database yang kita pakai di Android,

iOS, dan desktop. Langkah 8 sampai 10 membahas pola **offline-first**, yaitu cara menyusun aplikasi supaya database menjadi sumber utama dan jaringan hanya menyegarkannya.

### Setelah menyelesaikan modul ini, kalian bisa

| No | Kemampuan                                                                       | Dibahas di          |
|----|---------------------------------------------------------------------------------|---------------------|
| 1  | Membedakan kapan cukup preferensi, kapan butuh database, dan kapan berkas       | Langkah 1 dan 2     |
| 2  | Menulis tabel dan query di berkas .sq dan memakai kode Kotlin buatan SQLDelight | Langkah 3 sampai 7  |
| 3  | Menampilkan data terakhir saat offline dan menyimpan draf form secara otomatis  | Langkah 8 sampai 10 |

Capaian ini mendukung **CPMK0501**, dalam bentuk aplikasi yang menyimpan data lokal dan tetap berfungsi tanpa internet. Kemampuan kedua juga yang diamati di observasi praktik minggu ini.

### Yang perlu disiapkan

Proyek Laporkampus dari Modul 6. Sedikit pengetahuan SQL dari mata kuliah Basis Data sangat membantu: CREATE TABLE, SELECT, INSERT, DELETE, WHERE, dan ORDER BY. Uji hasilnya di emulator atau HP Android, karena di sana mode pesawat mudah dinyalakan.

#### **PENTING** Mengingat modul sebelumnya

Kita memakai expect dan actual dari Modul 1, Flow dan operator map dari Modul 2, ViewModel, StateFlow, dan copy dari Modul 4, stateIn dari Modul 5, serta LaporanApi dan fungsi keLaporan() dari Modul 6.

## PEMBUKA

## Masalah pembuka: sinyal hilang, laporan ikut hilang

Coba lakukan ini di emulator. Buka Laporkampus sampai daftar laporan tampil. Nyalakan mode pesawat, tutup aplikasi, lalu buka lagi. Yang tampil hanya pesan Tidak bisa terhubung, padahal semenit lalu daftarnya ada.

Lalu buka form, ketik judul Pintu macet dan lokasi GKU 2 R305. Tutup aplikasi dari daftar aplikasi terbaru, lalu buka form lagi. Isinya kosong.

**TEBAK DULU** **Coba pikirkan**

Bagaimana aplikasi tetap menampilkan data terakhir dan menyimpan draf, walau tidak ada internet?

**JAWABAN** **Arah jawabannya**

Datanya harus disimpan di HP. Pertanyaan berikutnya adalah menyimpan di mana dan dengan cara apa. Ada beberapa tempat menyimpan di perangkat, dan masing-masing cocok untuk jenis data yang berbeda. Kita bahas di Langkah 1.

## LANGKAH 1

## Tiga tempat menyimpan di perangkat

| Pilihan                                              | Cocok untuk                                              | Contoh di Laporkampus                    |
|------------------------------------------------------|----------------------------------------------------------|------------------------------------------|
| Preferensi key value, misalnya DataStore Preferences | Beberapa nilai kecil yang dibaca utuh                    | Tema gelap, nama pelapor, gedung favorit |
| Database, dengan SQLDelight                          | Banyak baris yang dicari, diurutkan, dan diubah sebagian | Daftar laporan, draf form                |
| Berkas                                               | Data besar yang tidak dicari isinya                      | Foto bukti laporan di Modul 8            |

Aturan praktisnya: kalau kalian butuh WHERE atau ORDER BY, itu database. Kalau hanya satu nilai dengan satu kunci, preferensi cukup. Foto disimpan sebagai berkas, lalu database hanya menyimpan alamat berkasnya.

**ANALOGI Kos-kosan**

Preferensi seperti catatan kecil yang ditempel di pintu kulkas: satu kalimat, langsung terbaca. Database seperti lemari arsip bersekat dengan label, sehingga satu berkas bisa dicari tanpa membongkar semuanya. Berkas seperti kardus di gudang untuk barang besar. Di lemari arsip cukup ditulis bahwa barangnya ada di kardus nomor sekian.

**PENTING Kenapa SQLDelight, bukan Room**

Di Android murni, banyak tutorial memakai Room. Room untuk Kotlin Multiplatform masih relatif baru. Di mata kuliah ini kita memakai SQLDelight, karena satu skema langsung berjalan di Android, iOS, dan desktop, dan dipakai di tutorial resmi Kotlin Multiplatform.

**LANGKAH 2**

## Preferensi, satu kunci dan satu nilai

commonMain/data/Pengaturan.kt

```
val TEMA_GELAP = booleanPreferencesKey("tema_gelap")

val temaGelap: Flow<Boolean> =
    datastore.data.map { it[TEMA_GELAP] ?: false }

suspend fun ubahTema(gelap: Boolean) {
    datastore.edit { it[TEMA_GELAP] = gelap }
}
```

| Bagian                             | Gunanya                                       |
|------------------------------------|-----------------------------------------------|
| <code>booleanPreferencesKey</code> | Kunci bertipe, jadi nilainya pasti Boolean    |
| <code>dataStore.data</code>        | Flow yang memancarkan nilai terbaru           |
| <code>?: false</code>              | Nilai bawaan kalau kunci belum pernah ditulis |
| <code>edit { }</code>              | Menulis secara aman, bertanda suspend         |

Objek `dataStore` dibuat sekali per platform, karena tiap platform punya lokasi berkas yang berbeda. Caranya ada di dokumentasi DataStore untuk Kotlin Multiplatform. Alternatif lain yang banyak dipakai adalah pustaka Multiplatform Settings. Kelompok bebas memilih, asal dipakai untuk nilai kecil saja.

Hands-on P7 di repo sengaja tidak memakai DataStore supaya pemasangannya lebih ringan. Latihan 1 menyimpan jumlah aplikasi dibuka di tabel Preferensi milik SQLDelight dengan pola yang sama: baca nilai lama, hitung yang baru, lalu tulis lagi.

#### TEBAK DULU

Dua ratus laporan disimpan di preferensi, semuanya sebagai satu teks JSON di bawah kunci daftar. Apa yang terjadi saat status satu laporan berubah? Dan saat kita ingin menampilkan laporan di GKU 1 saja?

#### JAWABAN

Untuk mengubah satu status, seluruh teks harus dibaca, diubah menjadi objek, diubah satu bagiannya, lalu ditulis ulang. Untuk menyaring per gedung, kedua ratus laporan harus dimuat dulu ke memori. Kalau aplikasi mati di tengah penulisan, seluruh daftar bisa rusak. Database menyelesaikan ketiganya: UPDATE untuk satu baris, WHERE untuk menyaring, dan transaksi supaya penulisan terjadi utuh atau tidak sama sekali.

#### ANALOGI **Absensi di satu lembar**

Menyimpan semua laporan di preferensi seperti menulis absensi satu semester di satu lembar kertas tanpa tabel. Untuk mengubah satu tanda hadir, kalian harus menyalin ulang seluruh lembar. Database seperti buku absensi bertabel: cukup mencentang satu kotak.

### LANGKAH 3

## Memasang SQLDelight

SQLDelight bekerja terbalik dari kebanyakan pustaka. Kita menulis SQL biasa, lalu SQLDelight membuatkan kode Kotlin yang aman tipenya. Pemasangannya mirip Ktor di Modul 6: satu API di commonMain, dengan driver yang berbeda di tiap platform.

```
composeApp/build.gradle.kts
```

```

plugins {
    alias(libs.plugins.sqlDelight)
}

kotlin {
    sourceSets {
        commonMain.dependencies {
            implementation(libs.sqlDelight.coroutines)
        }
        androidMain.dependencies {
            implementation(libs.sqlDelight.android.driver)
        }
        iosMain.dependencies {
            implementation(libs.sqlDelight.native.driver)
        }
        desktopMain.dependencies {
            implementation(libs.sqlDelight.sqlite.driver)
        }
    }
}

sqlDelight {
    databases {
        create("LaporKampusDb") {
            packageName.set("id.ac.itera.laporkampus.db")
        }
    }
}

```

| Bagian                                             | Artinya                                                                        |
|----------------------------------------------------|--------------------------------------------------------------------------------|
| Plugin <code>sqlDelight</code>                     | Membaca berkas .sq dan membuatkan kode Kotlin saat build                       |
| <code>coroutines</code> di <code>commonMain</code> | Pustaka <code>coroutines-extensions</code> , untuk mengubah query menjadi Flow |
| Driver per platform                                | Android driver, native driver untuk iOS, dan sqlite driver untuk desktop       |
| <code>create("LaporKampusDb")</code>               | Nama kelas database yang akan dibuat                                           |
| <code>packageName</code>                           | Paket tempat kelas buatan itu berada                                           |

Semua pustaka ada di grup `app.cash.sqlDelight`. Hands-on P7 memakai SQLDelight seri 2.0 dengan nama database AppDatabase. Samakan versi di `libs.versions.toml` untuk semua anggota kelompok.

#### LANGKAH 4

## Berkas .sq, tabel dan query

Tabel dan query ditulis di berkas berakhiran `.sq`. Letaknya di folder `commonMain/sqlDelight`, bukan di folder kotlin, dengan susunan folder yang mengikuti `packageName`.

```
commonMain/sqlDelight/id/ac/itera/laporkampus/db/Laporan.sq
```

```
CREATE TABLE LaporanEntity (
  id INTEGER NOT NULL PRIMARY KEY,
  judul TEXT NOT NULL,
  lokasi TEXT NOT NULL,
  status TEXT NOT NULL,
  petugas TEXT
);

semua:
SELECT * FROM LaporanEntity
ORDER BY id DESC;

byId:
SELECT * FROM LaporanEntity WHERE id = ?;

simpan:
INSERT OR REPLACE INTO LaporanEntity
VALUES (?, ?, ?, ?, ?);

hapusSemua:
DELETE FROM LaporanEntity;
```

| Bagian                         | Artinya                                           |
|--------------------------------|---------------------------------------------------|
| <code>semua:</code>            | Nama query, yang nanti menjadi nama fungsi Kotlin |
| <code>?</code>                 | Parameter yang diisi saat fungsi dipanggil        |
| <code>INSERT OR REPLACE</code> | Tambah baris baru, atau timpa kalau id sudah ada  |
| <code>ORDER BY id DESC</code>  | Laporan terbaru di atas                           |

Id di tabel ini sama dengan id dari server, jadi kita tidak memakai AUTOINCREMENT. Hands-on P7 memakai AUTOINCREMENT di tabel Catatan karena catatan dibuat di perangkat. Status disimpan sebagai teks, sama seperti di LaporanDto pada Modul 6.

#### TEBAK DULU

Kolom `petugas` tidak punya NOT NULL. Apa tipe `petugas` di kode Kotlin yang dibuatkan? Lalu, kalau di query simpan kita salah ketik nama tabel menjadi `LaporanEntiti`, kapan kesalahannya ketahuan?

#### JAWABAN

`petugas` bertipe **String?**, nullable, karena kolomnya boleh NULL. Kolom dengan NOT NULL menjadi String biasa, dan INTEGER menjadi **Long**, bukan Int. Salah ketik nama tabel ketahuan **saat build**. SQLDelight memeriksa setiap query terhadap skema tabel, jadi salah ketik nama tabel atau kolom membuat build gagal, bukan aplikasi crash di HP pengguna.

**ANALOGI Pemeriksa ejaan**

SQLDelight seperti pemeriksa ejaan yang membaca surat sebelum dikirim. Kesalahan ditandai saat kalian masih menulis, bukan setelah surat sampai ke penerima. Bandingkan dengan menulis SQL sebagai teks biasa di dalam kode, yang kesalahannya baru ketahuan saat aplikasi sudah dipakai.

**LANGKAH 5**

## Dari .sq menjadi Kotlin

Setelah build, SQLDelight membuat dua hal dari berkas Laporan.sql:

| Dari                                              | Menjadi                                                        | Isinya                                                    |
|---------------------------------------------------|----------------------------------------------------------------|-----------------------------------------------------------|
| <code>CREATE TABLE LaporanEntity</code>           | data class LaporanEntity                                       | Satu properti untuk setiap kolom, dengan tipe yang sesuai |
| Nama berkas Laporan.sql                           | Kelas LaporanQueries                                           | Satu fungsi untuk setiap query bernama                    |
| Query <code>simpan</code> dengan lima tanda tanya | Fungsi <code>simpan(id, judul, lokasi, status, petugas)</code> | Parameter bertipe Long, String, String, String, String?   |
| Query semua                                       | Fungsi <code>semua()</code>                                    | Mengembalikan objek Query berisi LaporanEntity            |

Bentuk data class buaatannya kira-kira seperti ini:

Hasil buatan SQLDelight, diringkas

```
data class LaporanEntity(
    val id: Long,
    val judul: String,
    val lokasi: String,
    val status: String,
    val petugas: String?
)
```

Kode aslinya ada di folder `build/generated/sqlDelight`. Boleh dibuka untuk dipelajari, tetapi jangan diedit, karena akan ditimpa setiap kali build.

**PENTING Kalau kelasnya belum muncul**

Kelas buatan SQLDelight baru ada setelah Gradle sync dan build. Kalau Android Studio menandai LaporanKampusDb atau LaporanQueries dengan merah, lakukan sync ulang atau jalankan task generate di panel Gradle.

**LANGKAH 6**

## Driver per platform lewat expect dan actual

Setiap platform punya cara sendiri untuk membuka database SQLite. Ini kesempatan pertama kita menulis expect dan actual sendiri, sejak dikenalkan di Modul 1.

commonMain/data/DriverFactory.kt

```
import app.cash.sqldelight.db.SqlDriver

expect class DriverFactory {
    fun createDriver(): SqlDriver
}
```

androidMain/data/DriverFactory.android.kt

```
actual class DriverFactory(private val context: Context) {
    actual fun createDriver(): SqlDriver =
        AndroidSqliteDriver(LaporanKampusDb.Schema, context, "laporkampus.db")
}
```

iosMain/data/DriverFactory.ios.kt

```
actual class DriverFactory {
    actual fun createDriver(): SqlDriver =
        NativeSqliteDriver(LaporanKampusDb.Schema, "laporkampus.db")
}
```

desktopMain/data/DriverFactory.desktop.kt

```
actual class DriverFactory {
    actual fun createDriver(): SqlDriver =
        JdbcSqliteDriver("jdbc:sqlite:laporkampus.db", Properties(),
        LaporanKampusDb.Schema)
}
```

Di commonMain kita berjanji ada kelas DriverFactory dengan fungsi createDriver. Tiap platform menepatinya dengan driver masing-masing. Versi Android butuh Context, jadi constructor-nya berbeda. Itu diperbolehkan, karena yang dijanjikan hanya kelas dan fungsinya.

**PENTING Driver desktop di repo**

Driver desktop di hands-on P7 memakai database di memori lalu memanggil `Schema.create` secara manual, sehingga data hilang saat aplikasi ditutup. Untuk proyek kelompok, pakai berkas seperti contoh di atas, dengan `Schema` sebagai parameter sesuai dokumentasi `SQLDelight`. Di Modul 8, pola `expect` dan `actual` yang sama kita pakai untuk kamera dan lokasi.

**LANGKAH 7**

## Membuat database dan memanggil query

commonMain/App.kt

```

val db = LaporanKampusDb(driverFactory.createDriver())
val q = db.laporanQueries

q.simpan(7, "AC mati", "GKU 1 R201", "Diproses", "Pak Joko")
q.simpan(8, "Kursi patah", "GKU 2 R305", "Baru", null)

val semua: List<LaporanEntity> = q.semua().executeAsList()
val satu: LaporanEntity? = q.byId(7).executeAsOneOrNull()

```

| Fungsi                            | Hasilnya                                      |
|-----------------------------------|-----------------------------------------------|
| <code>executeAsList()</code>      | Semua baris sekaligus                         |
| <code>executeAsOneOrNull()</code> | Satu baris, atau null kalau tidak ada         |
| <code>asFlow()</code>             | Flow yang ikut berubah saat isi tabel berubah |

Database dibuat sekali di tingkat aplikasi, sama seperti `HttpClient` di Modul 6. Di Android, `DriverFactory` dibuat di `MainActivity` karena butuh `Context`, lalu dikirim ke App. Perhatikan tipe parameter `simpan`: id bertipe `Long`, dan petugas boleh `null`, persis seperti tebakan di Langkah 4.

### Query yang diamati dengan Flow

`executeAsList` memblokir thread sampai selesai, jadi untuk layar kita pakai versi `Flow`.

commonMain/data/SumberLaporanOffline.kt

```

import app.cash.sqldelight.coroutines.asFlow
import app.cash.sqldelight.coroutines.mapToList

fun LaporanEntity.keLaporan() = Laporan(
    id = id.toInt(),
    judul = judul,
    lokasi = lokasi,
    fotoUrl = null,
    status = when (status) {
        "Diproses" → StatusLaporan.Diproses(petugas.orEmpty())
        "Selesai" → StatusLaporan.Selesai("")
        else → StatusLaporan.Baru
    }
)

fun pantauSemua(db: LaporkampusDb): Flow<List<Laporan>> =
    db.laporanQueries.semua()
        .asFlow()
        .mapToList(Dispatchers.Default)
        .map { daftar → daftar.map { it.keLaporan() } }

```

`asFlow()` memancarkan hasil query, lalu memancarkan lagi setiap kali isi tabelnya berubah.

`mapToList` menjalankan query di dispatcher yang diberikan dan mengubah hasilnya menjadi daftar.

`keLaporan()` mengubah `LaporanEntity` menjadi `Laporan`, sama seperti versi DTO di Modul 6, hanya id-nya diubah dari Long ke Int.

#### ANALOGI Kamera pengawas

`asFlow` seperti kamera pengawas yang terus menyorot satu rak. Siapa pun yang menaruh atau mengambil barang di rak itu, entah petugas form, petugas sinkronisasi, atau tombol hapus, layar pemantau langsung menampilkan isi rak terbaru. Tidak perlu ada yang menelepon untuk melapor.

#### LANGKAH 8

## Prinsip offline-first

Di Modul 6, layar membaca langsung hasil panggilan jaringan. Dengan **offline-first**, layar membaca dari database, dan jaringan hanya bertugas menyegarkan isi database.

|               | Modul 6                    | Offline-first                                  |
|---------------|----------------------------|------------------------------------------------|
| Layar membaca | Hasil panggilan jaringan   | Query database lewat Flow                      |
| Jaringan      | Satu-satunya sumber data   | Menyegarkan isi database                       |
| Saat offline  | Layar Gagal, daftar kosong | Data terakhir tetap tampil, dengan pesan kecil |

|           | Modul 6                   | Offline-first                   |
|-----------|---------------------------|---------------------------------|
| Draf form | Hanya di memori ViewModel | Disimpan otomatis ke tabel Draf |

Database menjadi **satu sumber kebenaran**. Siapa pun yang ingin mengubah apa yang tampil di layar, caranya selalu menulis ke database.

#### ANALOGI Papan pengumuman jurusan

Mahasiswa membaca papan pengumuman, bukan menelepon bagian akademik setiap kali ingin tahu jadwal. Petugas yang menempel pengumuman baru saat ada kabar. Kalau telepon ke akademik sedang putus, papan tetap berisi pengumuman terakhir. Papan itu database, dan petugasnya adalah fungsi segarkan.

#### LANGKAH 9

## Baca lokal, segarkan dari server

### Sumber offline

commonMain/data/SumberLaporanOffline.kt

```
class SumberLaporanOffline(
    private val db: LaporkampusDb,
    private val api: LaporanApi
) {
    fun pantauSemua(): Flow<List<Laporan>> = pantauSemua(db)

    suspend fun segarkan() {
        val dariServer = api.semua() // bisa gagal
        db.transaction {
            db.laporanQueries.hapusSemua()
            dariServer.forEach {
                db.laporanQueries.simpan(
                    it.id.toLong(), it.judul, it.lokasi, it.status, it.petugas
                )
            }
        }
    }
}
```

Ada dua hal penting di fungsi segarkan.

Pertama, **urutannya**. Ambil dulu dari server, baru tulis ke database. Kalau `api.semua()` gagal, exception keluar sebelum database disentuh, sehingga data lama aman. Kalau urutannya dibalik, yaitu hapus dulu baru ambil, maka saat offline database malah kosong.

Kedua, **transaction**. Hapus dan simpan terjadi sebagai satu kesatuan: semuanya berhasil, atau tidak ada yang berubah sama sekali. Flow pun hanya memancarkan sekali di akhir, bukan sekali untuk setiap baris.

#### ANALOGI **Transfer bank**

Transaksi database seperti transfer bank. Saldo pengirim berkurang dan saldo penerima bertambah sebagai satu kesatuan. Tidak boleh terjadi saldo pengirim sudah berkurang tetapi saldo penerima belum bertambah karena listrik padam di tengah jalan.

Menghapus semua lalu menyimpan ulang adalah cara paling sederhana dan cukup untuk proyek kita. Aplikasi besar biasanya hanya mengambil perubahan sejak waktu tertentu, tetapi idenya sama: jaringan menulis ke database, bukan ke layar.

### ViewModel: daftar dari database

commonMain/ui/DaftarViewModel.kt

```
class DaftarViewModel(private val sumber: SumberLaporanOffline) : ViewModel() {
    val laporan: StateFlow<List<Laporan>> = sumber.pantauSemua()
        .stateIn(viewModelScope, SharingStarted.WhileSubscribed(5000), emptyList())

    private val _pesan = MutableStateFlow<String?>(null)
    val pesan = _pesan.asStateFlow()

    init { segarkan() }

    fun segarkan() {
        viewModelScope.launch {
            _pesan.value = try {
                sumber.segarkan()
            } catch (e: CancellationException) {
                null
            } catch (e: Exception) {
                "Offline. Menampilkan data terakhir."
            }
        }
    }
}
```

`laporan` selalu dari database. `pesan` hanya hasil `segarkan`, bernilai null kalau berhasil. Tombol Coba lagi cukup memanggil `segarkan()`.

Bandungkan dengan `DaftarViewModel` di Modul 6. Keadaan Gagal tidak lagi menggantikan daftar. Ia turun pangkat menjadi pesan kecil di atas daftar. Keadaan memuat hanya benar-benar perlu saat database masih kosong di pemakaian pertama, dan boleh ditambahkan sebagai Boolean `menyegarkan`.

commonMain/ui/DaftarScreen.kt

```

@Composable
fun DaftarScreen(vm: DaftarViewModel, bukaDetail: (Int) → Unit, bukaForm: () → Unit)
{
    val laporan by vm.laporan.collectAsState()
    val pesan by vm.pesan.collectAsState()
    Column {
        pesan?.let { teks →
            Row(Modifier.fillMaxWidth().background(Color(0xFFF6EDCF)).padding(12.dp),
                verticalAlignment = Alignment.CenterVertically) {
                Text(teks, Modifier.weight(1f))
                TextButton(onClick = vm::segarkan) { Text("Coba lagi") }
            }
        }
        LayarDaftar(laporan, bukaDetail, bukaForm)
    }
}

```

**TEBAK DULU**

Mode pesawat menyala, lalu aplikasi ditutup. Setelah itu LaporanKampus dibuka lagi. Apa yang tampil di daftar? Kapan laporan baru dari petugas muncul?

**JAWABAN**

Yang tampil adalah **daftar terakhir yang tersimpan di database**, ditambah pesan Offline. Menampilkan data terakhir. Laporan baru muncul setelah internet kembali dan segarkan berhasil, entah karena tombol Coba lagi, layar dibuka ulang, atau tarik untuk menyegarkan yang kita bahas di Modul 13. Begitu segarkan menulis ke database, Flow memancarkan dan daftar berubah sendiri.

## Coba alurnya di Kotlin Playground

Database diganti MutableStateFlow supaya bisa dijalankan tanpa SQLDelight. Setiap elemen jawabanServer adalah hasil satu kali segarkan, dan null berarti mode pesawat.

Main.kt

```

import kotlinx.coroutines.*
import kotlinx.coroutines.flow.MutableStateFlow

val database = MutableStateFlow(listOf("AC mati"))
val jawabanServer = listOf(
    listOf("AC mati", "Kursi patah"),
    null,
    listOf("AC mati", "Kursi patah", "Pintu macet")
)

fun main() = runBlocking {
    val layar = launch { database.collect { println("Layar: $it") } }
    for (jawaban in jawabanServer) {
        delay(300)
        if (jawaban == null) println("Gagal: Offline. Menampilkan data terakhir.")
        else database.value = jawaban
    }
    delay(300)
    layar.cancel()
}

```

**TEBAK DULU**

Tulis keluarannya. Lalu, apa yang tampil kalau jawaban ketiga juga diganti menjadi null?

**JAWABAN****Keluaran**

```

Layar: [AC mati]
Layar: [AC mati, Kursi patah]
Gagal: Offline. Menampilkan data terakhir.
Layar: [AC mati, Kursi patah, Pintu macet]

```

Saat gagal, layar tidak mencetak apa pun karena database tidak berubah, tetapi daftar terakhir tetap tampil. Kalau jawaban ketiga juga null, layar berhenti di AC mati dan Kursi patah, yaitu data terakhir yang berhasil masuk database, dan baris Gagal tercetak dua kali.

**LANGKAH 10**

## Draf form tersimpan otomatis

Masalah kedua dari pembuka adalah draf yang hilang. Kita buat tabel kecil khusus draf.

```
commonMain/sqlDelight/id/ac/itera/laporkampus/db/Draf.sql
```

```

CREATE TABLE Draf (
  id INTEGER NOT NULL PRIMARY KEY,
  judul TEXT NOT NULL,
  lokasi TEXT NOT NULL
);

simpanDraf:
INSERT OR REPLACE INTO Draf VALUES (1, ?, ?);

ambilDraf:
SELECT * FROM Draf WHERE id = 1;

hapusDraf:
DELETE FROM Draf;

```

Hanya ada satu draf, jadi id selalu 1 dan INSERT OR REPLACE menyimpannya setiap kali disimpan.

commonMain/ui/FormLaporanViewModel.kt

```

@OptIn(FlowPreview::class)
class FormLaporanViewModel(
  private val sumber: Sumber<Laporan>,
  private val q: DrafQueries
) : ViewModel() {
  private val _uiState = MutableStateFlow(FormLaporanUiState())
  val uiState = _uiState.asStateFlow()

  init {
    q.ambilDraf().executeAsOneOrNull()?.let { d →
      _uiState.update { it.copy(judul = d.judul, lokasi = d.lokasi) }
    }
    viewModelScope.launch {
      _uiState.debounce(500).collect {
        q.simpanDraf(it.judul, it.lokasi)
      }
    }
  }
  // ubahJudul, ubahLokasi, dan kirim seperti Modul 6.
  // Setelah kirim berhasil: q.hapusDraf()
}

```

Saat ViewModel dibuat, draf lama dibaca dan dimasukkan ke form. Setelah itu, setiap perubahan uiState disimpan ke tabel Draf. `debounce(500)` menunggu pengguna berhenti mengetik setengah detik sebelum menulis, supaya database tidak ditulis di setiap huruf. `debounce` masih berlabel pratinjau di kotlinx.coroutines, sehingga butuh anotasi `@OptIn(FlowPreview::class)`.

#### ANALOGI Notulis rapat

debounce seperti notulis rapat yang menunggu pembicara selesai satu kalimat sebelum mencatat. Ia tidak menulis ulang catatan setiap kali pembicara mengucapkan satu kata, tetapi juga tidak menunggu sampai rapat selesai.

**PENTING Rapikan di proyek kelompok**

Di contoh ini ViewModel menerima DraftQueries langsung, supaya pola draf mudah dilihat. Di proyek, sebaiknya bungkus di dalam repository, supaya ViewModel tidak mengenal database secara langsung. Laporan yang dikirim saat offline sebaiknya tetap menjadi draf dengan pesan gagal kirim, bukan dibuang.

Coba sekarang: ketik judul, tutup aplikasi dari daftar aplikasi terbaru, lalu buka form lagi. Judulnya kembali.

## PRAKTIK

# Latihan mandiri

Ada tiga latihan di folder P7 Local Data Storage Hands-on. Semuanya bisa dikerjakan tanpa internet. Latihan 2 dan 3 yang diamati.

| Latihan | Berkas                  | Yang dilatih                                                            |
|---------|-------------------------|-------------------------------------------------------------------------|
| 1       | Handson1Preferensi.kt   | Jumlah aplikasi dibuka yang tetap berlanjut setelah aplikasi ditutup    |
| 2       | Handson2Repository.kt   | CatatanRepository dengan getAll sebagai Flow, tambah, dan hapus         |
| 3       | Handson3OfflineNotes.kt | LazyColumn catatan, form tambah, dan tombol hapus yang datanya bertahan |

Sync Gradle pertama membuat kelas AppDatabase, jadi buka proyeknya di awal sesi supaya sync selesai sebelum kalian mulai menulis kode.

## CONTOH Petunjuk latihan 1

Polanya sama dengan preferensi di Langkah 2: baca nilai lama dengan `executeAsOneOrNull()`, gunakan 0 kalau masih null, tambah satu, lalu simpan dengan query `INSERT OR REPLACE`.

## CONTOH Petunjuk latihan 2

`getAll` memakai pola Langkah 7: `asFlow()` lalu `mapToList(Dispatchers.Default)`. Fungsi tambah dan hapus cukup memanggil query, tanpa memanggil ulang `getAll`, karena Flow akan memancarkan sendiri setiap kali tabel berubah.

## CONTOH Petunjuk latihan 3

Layar membaca Flow dari repository dengan `collectAsState(initial = emptyList())`. Uji di Android: tambah catatan, tutup aplikasi, nyalakan mode pesawat, lalu buka lagi. Catatannya harus tetap ada. Untuk tantangan, ganti driver desktop menjadi berkas seperti Langkah 6 supaya datanya bertahan juga di desktop.

## Yang diamati

| Level       | Skor          | Yang terlihat                                                                 |
|-------------|---------------|-------------------------------------------------------------------------------|
| Sangat baik | 85 sampai 100 | Catatan bertahan setelah aplikasi ditutup dan tetap tampil dalam mode pesawat |
| Baik        | 70 sampai 84  | Tersimpan, tetapi hapus atau tampilan daftar belum benar                      |
| Cukup       | 55 sampai 69  | Data hanya di memori, hilang saat aplikasi ditutup                            |
| Kurang      | di bawah 55   | Belum ada penyimpanan lokal                                                   |

Observasi dikerjakan individu dengan bobot 4% dari nilai akhir, untuk CPMK0501. Buktinya: push latihan ke repository latihan pribadi, lalu kirim tautan commit di LMS sebelum kelas selesai. Uji di Android, karena driver desktop di repo memakai memori.

## Kalau error

| Gejala atau pesan                                            | Penyebab                                   | Yang dicoba                                                                  |
|--------------------------------------------------------------|--------------------------------------------|------------------------------------------------------------------------------|
| Unresolved reference: AppDatabase atau CatatanQueries        | Kode SQLDelight belum dibuat               | Sync ulang Gradle, atau jalankan task generateCommonMainAppDatabaseInterface |
| Data kosong lagi setiap dibuka di desktop                    | Driver desktop memakai memori              | JdbcSqliteDriver dengan berkas jdbc:sqlite                                   |
| no such table: Catatan                                       | Skema belum dibuat di driver desktop       | Panggil AppDatabase.Schema.create(driver), atau berikan Schema ke driver     |
| NotImplementedError: Lengkapi getAll()                       | Latihan 2 belum selesai                    | Selesaikan latihan 2 dulu                                                    |
| Unresolved reference: asFlow                                 | Dependency atau import belum ada           | Tambahkan coroutines-extensions dan import-nya                               |
| Build gagal dengan pesan No table found atau No column found | Nama kolom atau tabel di query salah ketik | Cocokkan dengan CREATE TABLE di berkas .sq                                   |

## PRAKTIK

## Bawa ke proyekmu

Kebutuhan Bank Proyek menyebut SQLDelight dengan draf offline. Bagi kerja: satu anggota skema dan driver, satu anggota sumber offline, satu anggota draf form. Layar daftar dan form hampir tidak berubah, karena tetap lewat ViewModel.

1. **Skema .sq.** Tabel entitas utama dengan query semua, byId, simpan, dan hapusSemua.
2. **Driver per platform.** expect dan actual DriverFactory untuk Android dan iOS atau desktop.
3. **Sumber offline.** Layar mengamati database, dan segarkan menulis hasil jaringan ke database.
4. **Draf form.** Tabel Draf yang terisi otomatis dan kembali saat form dibuka.

**CONTOH Contoh dari LaporanKampus**

Laporan.sq dan Draf.sq, DriverFactory untuk tiga platform, serta SumberLaporanOffline dengan pantauSemua dan segarkan. Dalam mode pesawat, daftar terakhir tetap tampil dengan pesan kecil di atasnya.

## Checkpoint minggu 7

| No | Yang diselesaikan                                                     |
|----|-----------------------------------------------------------------------|
| 1  | Skema .sq untuk data utama dan driver di setiap platform kelompok     |
| 2  | Layar membaca database, mode pesawat tetap menampilkan daftar         |
| 3  | Isi form tersimpan otomatis dan kembali setelah aplikasi dibuka ulang |

Push ke repository kelompok. README memuat tangkapan layar daftar saat mode pesawat dan penjelasan singkat alur offline-first. Checkpoint tidak masuk nilai, tetapi menjadi bekal tes lisan di pertemuan 12 sampai 16. Tes lisan 1 memeriksa lima layar, data REST, dan offline, jadi checkpoint yang tertunda sebaiknya dikejar sekarang.

## PENUTUP

# Rangkuman

---

Ada tiga tempat menyimpan di perangkat. **Preferensi** untuk beberapa nilai kecil dengan satu kunci. **Database** untuk banyak baris yang dicari, diurutkan, dan diubah sebagian. **Berkas** untuk data besar seperti foto, dengan alamatnya dicatat di database.

**SQLDelight** membaca tabel dan query SQL di berkas .sq, lalu membuat data class dan fungsi Kotlin yang aman tipenya. INTEGER menjadi Long, kolom tanpa NOT NULL menjadi nullable, dan salah ketik SQL ketahuan saat build. Driver dibuat per platform lewat expect dan actual.

`asFlow()` dan `mapToList` mengubah query menjadi Flow yang memancarkan ulang setiap kali tabel berubah.

**Offline-first** menjadikan database satu sumber kebenaran. Layar membaca database, dan jaringan hanya menyegarkannya. Segarkan selalu mengambil dari server dulu, baru menulis ke database di dalam transaction. Kegagalan jaringan menjadi pesan kecil, bukan pengganti daftar.

Draf form disimpan otomatis ke tabel Draf dengan `debounce`, dibaca kembali saat form dibuka, dan dihapus setelah laporan berhasil terkirim.

## PENUTUP

# Cek pemahaman

Jawab tanpa melihat modul dulu, lalu cocokkan dengan kunci di bawahnya.

1. Kapan cukup memakai preferensi, bukan SQLDelight?
2. Apa yang kita tulis di berkas .sq, dan apa yang dibuatkan SQLDelight?
3. Kolom `id INTEGER NOT NULL` dan `petugas TEXT` menjadi tipe apa di Kotlin?
4. Kenapa DriverFactory ditulis dengan expect dan actual?
5. Kenapa layar membaca dari database, bukan langsung dari jaringan?
6. Kenapa segarkan mengambil data dari server dulu, baru menghapus dan menyimpan ke database?
7. Apa gunanya `debounce(500)` pada penyimpanan draf?

### JAWABAN Kunci jawaban

1. Untuk nilai kecil dengan satu kunci, seperti tema, nama pengguna, atau gedung favorit, bukan untuk daftar data yang perlu dicari atau diurutkan.
2. Kita menulis CREATE TABLE dan query SQL bernama. SQLDelight membuatkan data class untuk tabel, kelas Queries, dan fungsi Kotlin untuk setiap query, lengkap dengan tipe parameternya.
3. id menjadi Long, dan petugas menjadi String? karena kolomnya boleh NULL.
4. Karena setiap platform membuka SQLite dengan driver yang berbeda. commonMain hanya berjanji ada fungsi createDriver, dan tiap platform menepatinya.
5. Supaya layar selalu punya data walau offline, dan hanya ada satu sumber kebenaran.
6. Supaya kalau server gagal, exception terjadi sebelum database disentuh, sehingga data lama tetap aman.
7. Supaya draf baru disimpan setelah pengguna berhenti mengetik setengah detik, bukan di setiap huruf.

## PENUTUP

## Bacaan lanjut dan persiapan minggu depan

| Sumber                                                               | Untuk apa                                       |
|----------------------------------------------------------------------|-------------------------------------------------|
| sqldelight.github.io, Getting Started with SQLite on Multiplatform   | Plugin, berkas .sq, dan driver per platform     |
| sqldelight.github.io, Coroutines                                     | asFlow, mapToList, dan mapToOneOrNull           |
| kotlinlang.org, Create a multiplatform app using Ktor and SQLDelight | Tutorial resmi yang menggabungkan Modul 6 dan 7 |
| developer.android.com, DataStore for Kotlin Multiplatform            | Preferensi di commonMain                        |
| developer.android.com, Offline-first app architecture                | Gambaran besar pola offline-first               |

**CONTOH Masalah Pertemuan 8**

Petugas fasilitas sulit mengecek laporan tanpa foto. Tombol Ambil foto bukti sudah ada di form, tetapi belum melakukan apa-apa. Bagaimana LaporKampus membuka kamera di Android dan iOS, dan apa yang terjadi kalau pengguna menolak izin? Bawa HP Android dan kabelnya kalau ada, karena kamera dan dialog izin terasa lebih nyata daripada di emulator.